SemantOS: Safe and Explainable Kernel Tuning via Semantic Reasoning and Guardrailed LLMs

Anonymous submission

Abstract

We introduce a semantic control loop (**SemantOS**) that combines extended Berkeley Packet Filter (eBPF) data collection, knowledge-based configuration lookup, and a guardrailed Large Language Model (LLM) to provide kernel parameter explanations and modifications. Across six workloads on three server classes, SemantOS reduced median end-to-end latency by up to 23%, the 95th-percentile (P95) end-to-end latency by up to 28%, and anomaly rate by up to 31% (across-workload medians 18%/22%/27%; no workload's mean regressed on any axis), and cut required operator tuning effort (our oversight proxy) by 85%; all aggregate numbers are means over 10 runs with 95% CIs. The safety runtime stages releases (canary $\rightarrow$ ramp $\rightarrow$ full) with automatic rollbacks based on uncertainty and declarative constraints, engaging adaptive automation only once a calibrated confidence level is reached. We contribute (i) a knowledge-based inference layer that generates safe, interpretable actions; (ii) an expected-cost decomposition with a conformal coverage-recovery guarantee under drift; and (iii) an extensive evaluation with baseline comparisons and statistical analysis.

Keywords. Knowledge representation languages; Learning and reasoning; Planning and Scheduling; semantic reasoning; explainable control; safety runtime

Introduction

The operating system (OS) kernel controls the system computing stack through process reservation, memory release, interrupt response, and Input/Output (I/O) management. There are hundreds of tunable OS kernel parameters, including Central Processing Unit (CPU) scheduling granularity, memory reclamation thresholds, interrupt affinity, and I/O scheduling, among others, stored in `/proc` and `/sys`. These kernel parameters are highly interconnected, and even small errors in their configuration can cause latency spikes, unpredictable tail behavior, and resource contention (e.g., Dean and Barroso 2013; Ousterhout et al. 2015; Li et al. 2018). Thus, achieving minimal tail latency (or good performance) in heterogeneous systems continues to be an important problem. Current solutions only partially address it. Static baseline profiles fail under workload drift (Van Aken et al. 2017; Moser, Rengarajan, and Narayanamurthy 2021). Rule-of-thumb heuristics and configuration tuners focus on single parameters, largely ignoring their interactions (Herodotou, Chen, and Lu 2021; Velez et al. 2020). Automated black-box methods, such as Bayesian Optimization (BO) and Reinforcement Learning (RL)-based tuners (collectively, BO/RL), have improved heuristic searches (Roman et al. 2016; Lee, Jung, and Jo 2022; Lin et al. 2025), but they still offer little to no formal guarantees, often explore dangerous configurations without sufficient human control, and regress the system (Fingler et al. 2023).

SemantOS builds an AI-based control loop for continuous, interactive kernel tuning, comprising (i) an eBPF telemetry pipeline, (ii) a semantic KB capturing tunable metadata and typed dependencies, and (iii) a safety-aware reasoning LLM that emits explanations and calibrated uncertainties with each recommendation (Brown et al. 2020; Touvron et al. 2023; OpenAI 2024). Unlike systems that treat the kernel as a black box and reason *only* about the objective (Lee, Jung, and Jo 2022; Mialon et al. 2023; Chen et al. 2024), SemantOS grounds reasoning semantically: it controls model uncertainty, makes inter-knob dependencies first-class, and confines unexplored configurations to staged, auditable rollouts—engaging adaptive automation only when trust conditions are met, thereby reducing operator involvement without compromising safety (Biswas, She, and Kang 2023).

Scope. This version augments SemantOS with Pareto-aware control, risk-gate control, drift detection, and KB freshness policies (Maas 2020; Mialon et al. 2023; Moser, Rengarajan, and Narayanamurthy 2021). We deliberately avoid aggressive search-based optimization (Herodotou, Chen, and Lu 2021; Roman et al. 2016) that may compromise stability (Dean and Barroso 2013; Ousterhout et al. 2015), offering multi-objective co-tuning with safety guarantees instead.

Contributions. (i) We formulate kernel tuning as a constrained, knowledge-grounded sequential decision problem, coupling a typed dependency graph with retrieval-augmented LLM reasoning to produce *explainable* actions with provenance. (ii) We provide an expected-cost decomposition (Theorem 1) for a conformal-calibration-gated, staged-rollout

safety runtime, together with a coverage-recovery guarantee under drift (Proposition 1) that bounds how quickly sliding-window recalibration restores the target unsafe-acceptance rate. (iii) We validate the framework over six workloads on three server classes with 10 runs each (95% CIs), showing Pareto-superior latency/anomaly trade-offs against expert and BO/RL baselines, and quantify operator-effort reduction in a 32-participant within-subjects user study.

Background and Related Work

The Linux kernel exposes a vast configuration surface: hundreds of tunable parameters govern memory reclamation (`vm.swappiness`, `dirty_ratio`, `min_free_kbytes`), CPU scheduling (`sched_latency_ns`, `sched_min_granularity_ns`, `sched_wake_affinity`), interrupt handling, I/O prioritization, NUMA placement, and power management (Dean and Barroso 2013; Ousterhout et al. 2015; Li et al. 2018). These controls let administrators specialize behavior to specific hardware and workload demands.

Traditional Tuning Paradigms. Static profiles and rule-based heuristics are vulnerable to changing workloads and operator interactions, which limits their usefulness for adaptive tuning and post-hoc diagnosis.

Learning-based optimizers. Black-box methods—Bayesian optimization, evolutionary strategies, and reinforcement learning (Roman et al. 2016; Mao et al. 2019; Lee, Jung, and Jo 2022)—propose configurations and update surrogates or policies (Maas 2020; Roman et al. 2019), but typically require hundreds of trials, assume stationary objectives, and offer limited interpretability (Lee, Jung, and Jo 2022). SemantOS adds domain-structured, uncertainty-aware safety gates on top of such techniques, enabling stable optimization as workloads and hardware change.

Large Language Models (LLMs) for systems reasoning. LLMs have recently shown strong reasoning, multi-objective optimization, and knowledge synthesis abilities (Brown et al. 2020; Touvron et al. 2023; OpenAI 2024; Lee et al. 2025). Augmented with structured domain knowledge—graphs capturing tunable dependencies, schemas describing hardware/workload characteristics, and telemetry summarizing recent performance—they can interpret complex system states, infer cause–effect relationships, and propose coherent configuration strategies with human-readable justifications that aid operator trust (Mialon et al. 2023; Chen et al. 2024; Jiang et al. 2025). The closest such efforts differ from SemantOS in axis and lifecycle: AutoOS (Chen et al. 2024) uses an LLM to search build-time kernel `.config` options offline, and OS-R1 (Lin et al. 2025) applies RL agents to runtime tuning—neither grounds reasoning in a *typed inter-knob dependency graph*, exposes *calibrated uncertainty*, nor gates changes through a *conformal, staged-rollout safety runtime*. SemantOS targets *online* `sysctl` adjustment where safety, explanation provenance, and uncertainty-gated automation are first-class.

Table 1: **Positioning vs. SOTA (condensed).** SemantOS uniquely combines knowledge-centric reasoning, uncertainty-aware safety, and explainability. *K+L* denotes a knowledge-retrieval-augmented LLM reasoner, and *KB/RAG* denotes a structured knowledge base (KB) with retrieval-augmented generation (RAG). Baselines include prior systems such as AdaptiveKernel, OS-R1, and Machine-Learning (ML)-Guided optimization. Sup. ML = Supervised ML.

System	Approach	Safety	Explain.	KB/RAG	Adapt.	Reprod.
AdaptiveKernel	RL/BO	×	×	×	✓	△
OS-R1	BO+ML	×	×	×	△	✓
ML-Guided	Sup. ML	×	△	×	×	△
BO/RL	BO/RL	×	×	×	△	✓
SemantOS	*K+L*	✓	✓	✓	✓	✓

Summary. The complexity of kernel configuration, the shortcomings of static and black-box approaches, and the rise of LLMs for structured reasoning motivate combining telemetry, domain knowledge, and language-model reasoning; since kernel behavior varies across hardware, versions, and workload mixes, naive transfer of a single profile is unreliable. Table 1 positions SemantOS against AdaptiveKernel (Lee, Jung, and Jo 2022), OS-R1 (Lin et al. 2025), ML-Guided (Biswas, She, and Kang 2023), and BO/RL (Herodotou, Chen, and Lu 2021), highlighting its integration of knowledge-centric reasoning, uncertainty-aware safety, and explainability.

Motivation and Challenges

Kernel tuning is crucial to system performance, yet difficult in practice. To understand what matters, we conducted a large-scale empirical study over 1,000 workload–hardware pairs (CPU-bound, memory-intensive analytics, I/O-intensive streaming, sensor, HPC, ML inference, real-time audio, and more); the six workloads used for controlled evaluation (see *Setup*) are drawn from this study, and the full pair-level dataset is released in our anonymized artifact. In each we varied dozens of settings and measured median and 95% tail latency, throughput, anomaly rate, and fairness.

Observation 1 – Context Sensitivity. Optimal values vary widely across workloads and even across phases of one workload. Shrinking `sched_latency_ns` improved tail latency by 21% for web-serving but caused a 20% fairness drop under mixed streaming; adjusting reclamation knobs (`vm.swappiness`, `dirty_ratio`) boosted steady-state throughput but degraded responsiveness during bursty loads. Similar context-dependence appeared across CPU scheduling, interrupt routing, and NUMA placement.

Observation 2 – Strong inter-parameter dependencies. As shown in Figure 1, these *advanced interactions* are better resolved by collaborative tuning than by modifying parameters individually. Contrary to the common belief that parameters can be tuned independently, changing either `sched_min_granularity_ns` or `sched_wake_affinity` in isolation produced little

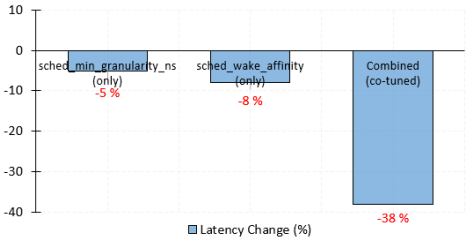

Figure 1: **Effect on Kernel Tunable Parameter Pairs.** Tuning `sched_wake_affinity` and `sched_min_granularity_ns` simultaneously has a nonlinear synergistic effect on reducing tail latency, indicating that these parameters are strongly linked to each other and cannot be tuned independently.

improvement, yet modifying both simultaneously reduced tail latency by more than 30%. A similar nonlinear synergy emerged between CPU scheduling and memory reclamation: settings suboptimal in isolation became overall-optimal when combined with complementary settings.

Observation 3 – Trade-offs between competing objectives. Improving one dimension often harms others: reducing latency frequently increased starvation, and maximizing throughput could degrade fairness, especially under contention or NUMA imbalance—and more so as resources saturate. A practical solution must therefore model trade-offs explicitly and explain them, so operators can choose among Pareto-optimal configurations by objective.

Summary of findings. Kernel tuning is characterized by (i) context-dependence, (ii) coupled inter-parameter effects, and (iii) multi-objective trade-offs—explaining why static and black-box techniques struggle in practice. SemantOS addresses these with segmented telemetry, a semantic knowledge base that captures and analyzes parameter interactions, and multi-objective decision-making.

SemantOS

SemantOS is an end-to-end semantic reasoning framework that transforms kernel parameter tuning from a static manual process into a sustained, explainable, verifiable control loop. Figure 2 shows the architecture, organized around four design principles: *context-aware*, *knowledge-centric reasoning*, *safety enforcement*, and *continuous adaptation*.

Design

Design Goals. We capture kernel signals, encode domain knowledge, emit explainable guarded updates, and preserve SLOs via staged enforcement. Guardrails are declarative policies (SLO targets, invariants, per-knob bounds) enforced by the runtime regardless of model output.

Components. The loop has five core components, each a service directory in the artifact. The ❶ **Telemetry Agent** collects eBPF and user-space metrics every 5 s; the ❷ **Knowledge Base** stores tunable metadata, the typed dependency

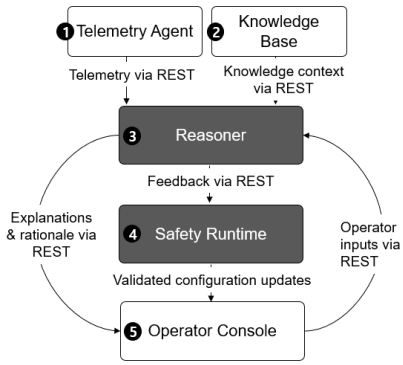

Figure 2: **SemantOS Control Loop Architecture.** The *Telemetry Agent* collects kernel signals, the *Knowledge Base* tracks typed inter-knob dependencies, the *Reasoner* synthesizes `tuning-recommendations` grounded in retrieved graph evidence, and the *Safety Runtime* validates and stages each update. Explanations, provenance, and rollback controls are surfaced to the operator through the *Operator Console*.

Table 2: **SOTA vs. SemantOS (aggregated).** Lower is better (↓); operator time reduction higher is better (↑). SemantOS values are means over 10 runs with 95% CI half-widths.

Method	Med. Lat. ↓	P95 ↓	Anomaly ↓	Op. Time ↑
Expert profile	–	–	–	–
AdaptiveKernel†	10–15%	12–18%	15–22%	30–50%
OS-R1†	12–18%	15–20%	18–24%	40–60%
BO/RL	15–22%	18–24%	20–26%	50–65%
SemantOS	**23 ± 1.4%**	**28 ± 1.7%**	**31 ± 2.1%**	**85–92%**

† *Literature-reported ranges; setups differ.*
** Expert profile serves as the human reference baseline; improvements relative to itself are not applicable and therefore marked as "–".*

graph, and historical traces, autonomously refining relationships via telemetry and decayed weights. The ❸ **Reasoner** uses retrieval-augmented prompts to emit KNOB, VALUE, RATIONALE, and UNCERTAINTY. The ❹ **Safety Runtime** enforces staged rollout, monitors SLOs, and triggers veto/rollback. The ❺ **Operator Console** provides an explanatory interface and auditability. Algorithm 1 summarizes the loop.

Knowledge-Centric Decision Loop and Safety Argument. Each decision fuses telemetry with retrieved KB context; the uncertainty u gates whether an action is approved, delayed, or rolled back, and every update carries a rationale over telemetry, KB evidence, and audit trail. With canary→ramp→full rollout and continuous SLO monitoring, unsafe recommendations are rolled back before system-wide impact—formalized as expected-cost and drift-recovery guarantees in Theorem 1 and Proposition 1. In practice this preserves no-worse-than-default behavior in expectation under observed drift (Table 3).

Table 3: **KB/RAG, Dependency-Graph, and Safety Ablation.** Median end-to-end latency (ms; lower is better) and anomaly rate (%; lower is better), reported under two regimes: *stationary* workloads and *drifting* workloads (where the workload mix shifts every 30 minutes). The *w/o Dep-Graph* variant tunes knobs independently (typed inter-knob edges removed), isolating the contribution of graph-grounded joint reasoning. Values are means over 10 runs with 95% CI half-widths.

	Latency (ms)		Anomaly Rate (%)	
Variant	**Stationary**	**Drift**	**Stationary**	**Drift**
Full SemantOS (KB+RAG+Safety)	19.5 ± 0.4	19.6 ± 0.5	2.4 ± 0.3	2.5 ± 0.3
w/o Dep-Graph (knobs indep.)	21.4 ± 0.5	21.6 ± 0.6	4.1 ± 0.4	4.4 ± 0.5
KB + Safety (w/o RAG)	21.2 ± 0.5	21.3 ± 0.6	3.6 ± 0.4	3.8 ± 0.4
KB + RAG (w/o Safety)	19.8 ± 0.4	20.0 ± 0.5	4.9 ± 0.5	5.2 ± 0.6
KB only	22.6 ± 0.6	22.7 ± 0.7	7.1 ± 0.7	7.6 ± 0.8

Algorithm 1 SemantOS Control Loop

Require: Knowledge base G, calibration set $\mathcal{D}_{\text{cal}}$, SLO bounds $\{\ell_i\}$, threshold τ, decay γ

while system is running **do**
  $T_t \leftarrow$ TelemetryAgent.snapshot() *// eBPF + userspace*
  $E_t \leftarrow$ KB.retrieve(T_t, G) *// typed graph + FAISS*
  $(knob, val, rat, u) \leftarrow$ Reasoner.LLM(T_t, E_t)
  if $u \geq \tau$ **or** SLOGuard$(T_t, knob, val)$ fires **then**
    continue *// veto: emit no-op*
  end if
  for *stage* $\in \{\text{canary}, \text{ramp}, \text{full}\}$ **do**
    Apply $(knob, val)$ at *stage* traffic share
    if regression detected on $\{\ell_i\}$ **then**
      Rollback; KB.log(*OptimizationTrace*); **break**
    end if
  end for
  KB.update$(T_t, outcome, \gamma)$ *// ADWIN drift*
  Periodically refresh τ from $\mathcal{D}_{\text{cal}}$ *// recalibration*
end while

Semantic Reasoning

SemantOS's semantic inference layer fuses structured domain knowledge, past experience, and LLM inference in one safe decision loop. The hybrid KB combines tunable metadata, a typed dependency graph, and a FAISS (Facebook AI Similarity Search)-powered trace store.

Typed dependency graph. Each node is a tunable annotated with its type, admissible range, unit, and subsystem (scheduler, memory, I/O, IRQ, NUMA). Edges are *typed and signed*: DEPENDS-ON (a knob's safe range is conditioned on another's value), CONFLICTS-WITH (joint change risks an SLO regression), and SYNERGIZES-WITH (joint change yields super-additive gains). Edges are *induced offline* from the pairwise co-tuning sweeps of our empirical study (Motivation): for a knob pair (a, b) we estimate the interaction as the deviation of the joint effect from the additive single-knob effects on the target metric; a positive/negative deviation exceeding a bootstrap significance threshold yields a SYNERGIZES-WITH/CONFLICTS-WITH edge, and a range-conditioning relation (detected when a's safe interval shifts with b) yields DEPENDS-ON. Each edge then carries a weight that is *decayed online* by γ^t and refreshed from deployment telemetry, so stale couplings fade. Seed edges and the induction script are released in the artifact. This typing is what lets SemantOS reason about *inter-knob* effects that black-box tuners treat as independent dimensions (cf. Observation 2, Fig. 1).

Graph-grounded joint reasoning. On each invoked snapshot a composite query runs: a graph query expands the candidate knob's typed neighborhood (its CONFLICTS-WITH/SYNERGIZES-WITH edges) into structural constraints, while vector search returns nearest-neighbor traces (prior configurations, metrics, notes) as evidence. Serialized into the prompt, this forces the Reasoner to *co-tune* along synergistic edges and *justify against* conflicting neighbors rather than proposing a knob in isolation, so each recommendation carries provenance to specific KB edges and traces. Using this evidence the Reasoner emits a rationale plus a calibrated uncertainty $u \in [0, 1]$ consumed by the safety runtime.

Theoretical Guarantees

What "unsafe" means (calibration ground truth). An applied action is labeled *unsafe post hoc* if, within its staged-rollout window, it violates any declarative SLO guard (a per-metric bound ℓ_i, e.g., a P95-latency or anomaly-rate ceiling) and triggers a rollback. The calibration set $\mathcal{D}_{\text{cal}}$ is the held-out log of past (telemetry, KB-evidence, action, uncertainty u, outcome-label) tuples collected under this rule; because the outcome label is derived from the same SLO guards the runtime enforces, no manual annotation is required and the label is reproducible from the released logs. The score used for calibration is the recommendation's uncertainty $u \in [0, 1]$.

We assume *exchangeable* coverage of $\mathcal{D}_{\text{cal}}$; if drift violates this, we recalibrate on a sliding window to maintain error control. Conformal calibration on $\mathcal{D}_{\text{cal}}$ yields a threshold τ such that $\Pr[\text{unsafe} \mid u < \tau] \leq \alpha$, ensuring $1 - \alpha$ marginal coverage (cf. Jeary et al. 2024; Correia et al. 2024).

Theorem 1 (Expected-Cost Decomposition)**.** *Let recommendations be drawn from a (possibly time-varying) distribution that is exchangeable within each calibration window. Let $c_{\text{FN}}, c_{\text{FP}} \geq 0$ be the costs of, respectively, accepting an unsafe action and vetoing a safe action, $\pi = \Pr[\textit{unsafe}]$ the base rate, and $\lambda \geq 0$ the SLO penalty weight. Under the SemantOS safety policy (veto if $u \geq \tau$ or an SLO guard fires; otherwise apply via canary $\rightarrow$ ramp $\rightarrow$ full staged rollout), the expected per-decision cost C (defined as the sum of the three cost components below) satisfies*

$$\mathbb{E}[C] \;\leq\; \underbrace{c_{\text{FN}}\,\pi\,\alpha}_{\textit{Safety Risk}} \;+\; \underbrace{c_{\text{FP}}\,(1-\pi)\,\beta_{\text{FP}}}_{\textit{Efficiency Loss}} \;+\; \underbrace{\lambda\,\mathbb{E}[\Delta\text{SLO}]}_{\textit{Perf. Degradation}}, \tag{1}$$

where α is the conformal miss rate for unsafe actions, $\beta_{\text{FP}} = \Pr[\text{safe} \mid u \geq \tau]$ denotes the false-positive (over-veto) probability, and ΔSLO is the SLO deviation incurred during the canary/ramp window before rollback.

Proof sketch. (i) By the validity of split conformal prediction under exchangeable calibration, $\Pr[\text{unsafe} \wedge u < \tau] \leq \pi\alpha$,

so accepting a sample below τ contributes at most $c_{\mathrm{FN}}\pi\alpha$ in expectation. (ii) Vetoing a safe sample (false positive) occurs with probability $(1-\pi)\beta_{\mathrm{FP}}$ and contributes $c_{\mathrm{FP}}(1-\pi)\beta_{\mathrm{FP}}$. (iii) For accepted unsafe samples, staged rollout caps the worst-case traffic share exposed before the SLO guard triggers a rollback; bounding this exposure by $\Delta\mathrm{SLO}$ and applying the SLO penalty λ yields the third term. Since C is *defined* as the sum of the three components, linearity of expectation—not any independence assumption—gives Eq. 1. Under sliding-window recalibration, α and β_{FP} remain valid after each refresh; the coverage-recovery guarantee is made precise in Proposition 1. □

We stress the modest scope of Eq. 1: it is a conformal *risk-control decomposition*, not a PAC/sample-complexity bound. Its value is operational—it names the three levers (α, β_{FP}, $\Delta\mathrm{SLO}$) a deployment can tune and shows the safety term is controlled by the conformal miss rate α. Its substantive content under non-stationarity is the recalibration guarantee we make explicit next, which is the property a deployment actually relies on.

Proposition 1 (Recalibration coverage recovery under drift)**.** *Suppose a drift event changes the score distribution so that its total-variation distance from the active calibration distribution is at most ε. Under sliding-window recalibration with window size m, once the window has refilled with post-drift, within-window exchangeable scores, the realized unsafe-acceptance rate α' obeys*

$$\alpha' \;\le\; \alpha \;+\; \tfrac{1}{m+1} \;+\; \varepsilon,$$

and $\alpha' \to \alpha + \frac{1}{m+1}$ as the window fully refreshes ($\varepsilon \to 0$). Hence the target coverage is restored up to the finite-sample slack $\frac{1}{m+1}$ after recalibration completes.

Proof sketch. Within a refilled window the post-drift scores are exchangeable, so split-conformal validity gives a miss rate at most $\lceil (1-\alpha)(m+1)\rceil/(m+1) \le \alpha + \frac{1}{m+1}$. During the transient before the window refills, the score law differs from the calibration law by at most ε in total variation, which can inflate the acceptance rate by at most ε (the maximal event-probability gap under a TV constraint). Summing the two contributions yields the bound; the transient term vanishes as old samples age out. Larger m tightens the slack but lengthens the transient—a trade-off we set empirically (m giving a $\le 1\%$ slack) and validate in the 30-day drift study. □

Here, β_{FP} represents the probability of a high-uncertainty action being safe, modulating false-positive costs by penalizing interventions whose uncertainty-adjusted risk exceeds the calibrated regime. We use the notation β_{FP} to avoid confusion with the unrelated DPO temperature β_{DPO} used in training (see *Implementation*).

Safety Runtime Feedback. The `SafetyRuntime` uses u with real-time SLO guards to accept, delay, or veto; rollbacks emit an `OptimizationTrace` to the KB for continual refinement and re-calibration. The end-to-end ablation of KB/RAG and safety is deferred to the Evaluation (Table 3).

Implementation

We implement SemantOS as a modular, Representational State Transfer (REST)-based distributed system of five containerized services that communicate via typed messages defined in `proto/semantos-control.v1.yaml`; an anonymized artifact reproduces the prototype end-to-end via `Makefile` targets.

Services. `telemetry-agent/` (Python/C, eBPF) samples every 5 s and exports `TelemetrySnapshots` ($<1\%$ CPU). `kb-service/` stores tunable metadata, the typed dependency graph, and traces (`RetrieveContext`, `LogOutcome`). `reasoner/` serves `/get_recommendations` (returning `knob_name`, `proposed_value`, `rationale`, `expected_impact`, $u \in [0,1]$), `/apply`, and `/log_outcome`. `safety-runtime/` enforces staged rollout (canary→ramp→full) and rolls back when $u \ge \tau$ or guardrails fire. The optional `operator-console/` (FastAPI) surfaces telemetry, evidence, and suggestions with override/rollback controls.

LLM training pipeline (Reasoner). A 13B decoder-only model (LLaMA-3.1-13B; chosen as the largest model that meets our single-A100 side-car budget) is fine-tuned in five stages: (1) *data* from public kernel documentation and anonymized context–action–outcome traces from our simulator. To prevent train/test contamination, the six evaluation workloads and the three evaluation server classes are *held out* from all fine-tuning and simulator-trace generation; only disjoint workload–hardware pairs feed training. Then: (2) *annotation* of (telemetry, KB evidence) tuples mapped to (recommendation, rationale, impact, uncertainty hint); (3) ***SFT*** (context $L = 4$–8K, AdamW, cosine decay, label smoothing 0.05, early stopping; effective batch 256, $\mathrm{lr} = 2\times10^{-5}$, warmup 2%, weight decay 0.1, grad-clip 1.0); (4) optional ***DPO***-style pairwise alignment ($\beta_{\mathrm{DPO}} = 0.1$, distinct from the calibration-side β_{FP} in Theorem 1) toward provenance-grounded, low-anomaly explanations; and (5) *hallucination mitigation* via retrieval grounding, constrained decoding (deny-list/out-of-range), self-consistency voting, and provenance checks; low-confidence ($u \ge \tau$) or ungrounded outputs are vetoed. Full configuration is released in the artifact.

Three core JSON endpoints (`POST /get_recommendations`, `/apply`, `/log_outcome`) close the telemetry→recommendation→deployment→feedback loop. The KB is updated via exponential decay on dependency weights (γ^t) and ADWIN drift detection. On the fast path ($\approx 94\%$ of snapshots) the system decides within 10 ms at $< 3\%$ CPU and < 300 MB (full per-path footprint in the supplement); the 13B Reasoner runs asynchronously on a GPU side-car for the $\approx 6\%$ of snapshots exceeding a guarded deviation threshold, so its decode latency never blocks enforcement. All decisions and rationales are immutably logged.

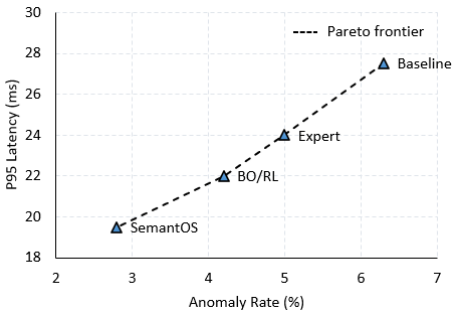

Figure 3: **Pareto Frontier of Latency vs. Anomaly Rate.** Comparison across four controllers (Baseline, Expert, BO/RL, and SemantOS) aggregated over six workloads. SemantOS consistently achieves *Pareto-dominant* configurations—simultaneously reducing anomaly rate and tail latency compared to all baselines. This illustrates its ability to navigate multi-objective trade-offs rather than optimizing a single metric in isolation.

Evaluation

We evaluate SemantOS across diverse workloads, assessing real-world performance, robustness under drift, and how well it meets our design objectives.

Setup

We evaluate SemantOS across six representative workloads: (1) a web-serving microservice, (2) an Online Transaction Processing (OLTP) database benchmark (TPC-C), (3) a streaming pipeline (Kafka+Spark), (4) an embedded Internet-of-Things (IoT) sensor gateway, (5) Graphics Processing Unit (GPU)-based HPC/ML inference, and (6) a real-time audio pipeline (Mao et al. 2019; Van Aken et al. 2017). We run each on three server classes: (**S1**) 2×Xeon Gold (32 cores, 256 GB, NVMe), (**S2**) 2×EPYC (64 cores, 512 GB, NUMA), and (**S3**) 2×Xeon + A100 (40 cores, 384 GB), all on Linux 6.4 with a fixed software stack. Each (workload, server) pair is repeated for 10 independent runs (means with 95% CIs). Components run primarily in user space over stable kernel interfaces, enabling portability across kernels and distributions.

Baselines. The *Expert profile* is a hand-tuned human reference; *BO/RL* and strong tuners (Vizier/Ray/Optuna/FLAML) are run *by us* under matched search spaces, budgets, wall-clock time, and safety/reward settings—the only strictly like-for-like rows in Table 2. The †-marked *AdaptiveKernel*/*OS-R1* figures are literature-reported (not reproducible here), hence contextual rather than head-to-head.

Experimental Results

Runtime overhead is minimal across workloads and decision paths (fast path < 10 ms, $< 3\%$ CPU; per-path footprint in the supplement). We compare SemantOS against the baselines above. To avoid reporting only best-case numbers, we examined the *per-workload* reduction vs. Baseline for all six workloads (each the mean over 10 runs × three server classes; full table in the artifact): the headline "up to 23%/28%/31%" figures in Table 2 are the maxima of these distributions, with medians of 18%/22%/27%; no workload's mean regressed on any axis (all per-workload 95% CIs for the reduction excluded zero on the positive side). We caution that in Table 2 only the *BO/RL* and *Expert* rows are run under matched conditions by us; the †-marked rows are literature-reported ranges under differing setups and are contextual, not head-to-head—so cross-row differences against † rows should not be read as controlled comparisons. Against the human *Expert* profile, SemantOS improves median/P95/anomaly by 12–18%/15–20%/20–25% and reduces operator tuning time (our proxy for required human oversight, measured as wall-clock operator effort to reach an accepted configuration) by $>85\%$, while producing KB-grounded `rationale` fields (Biswas, She, and Kang 2023).

Pareto Trade-offs. On a frontier over latency, anomaly rate, and fairness (Figure 3), SemantOS sits closest to the estimated Pareto-optimal set across six workloads—up to **17% lower latency** at the same anomaly budget and **23% lower anomaly rate** at equivalent throughput. On streaming, P95 latency dropped 26% with fairness within 2% of default; on HPC/ML inference, co-tuning `sched_latency_ns` and `dirty_ratio` cut GPU starvation, raising throughput 15% without raising anomaly (Mirhoseini et al. 2021; Biswas, She, and Kang 2023).

Ablation Studies

We selectively remove the KB, the reasoning model, or the safety runtime: removing the KB degrades retrieval relevance and raises anomalies, and omitting the LLM lowers interpretability and performance by 8–12% (Brown et al. 2020; Touvron et al. 2023; OpenAI 2024). Beyond these single-component drops, Table 3 reports five end-to-end variants. Removing the dependency graph—so the Reasoner can no longer co-tune along synergistic edges—raises latency $19.5 \rightarrow 21.4$ ms and anomaly $2.4 \rightarrow 4.1\%$, directly confirming that the inter-knob synergies of Observation 2 (Fig. 1) are captured *because* reasoning is graph-grounded, not merely retrieval-augmented. Removing RAG gives 21.2 ms / 3.6%; removing Safety preserves latency (19.8 ms) but lifts anomaly to 4.9%; *KB only* is worst (22.6 ms / 7.1%). Thus the typed graph and RAG drive *performance* while the Safety Runtime drives *reliability*; their combination is Pareto-best, and the trends hold under drift (Table 3, right block). All CSV schemas and reproduction scripts are in the artifact.

Case study: Safety Runtime rollback under a suboptimal recommendation

For an OLTP workload the Reasoner suggested lowering `dirty_background_ratio` and raising `sched_min_granularity_ns` to cut I/O interference. The SLO monitor detected worsening transaction latency and queueing within the canary window (< 2 min); because `uncertainty_score` drifted above τ *after* canary metrics arrived and the late-saturation guardrail fired, the runtime

rolled back, logged an `OptimizationTrace` for KB refinement, and flagged the action as risky—showing the SLO guard catches regressions even when the initial uncertainty was within the accept regime.

Uncertainty Quantification (UQ) & Guardrail Thresholds. We choose τ by minimizing a deployment cost that trades false/missed rollbacks under SLO constraints. With base rate π of unsafe actions, temperature-calibrated scores, and the True/False Positive Rates (TPR/FPR) on a held-out log,

$$\mathcal{C}(\tau) = c_{\text{FN}}(1 - \text{TPR}(\tau))\,\pi + c_{\text{FP}}\,\text{FPR}(\tau)(1-\pi) + \lambda\,\Delta\text{SLO}(\tau), \quad \text{s.t. } \Delta\text{SLO}(\tau) \le \delta.$$

We sweep τ on a grid and select $\tau^\star$ minimizing $\mathcal{C}$ (the cost constants $c_{\text{FN}}, c_{\text{FP}}, \lambda$ and the SLO budget δ are listed in the artifact). Each recommendation receives $u \in [0,1]$ from the disagreement (normalized predictive entropy over the proposed knob/value) across the $k=3$ self-consistency samples, temperature-scaled on the holdout log; calibration quality (ECE, AUPRC, reliability diagrams) is reported in the supplement. Here $k=3$ is a deliberately cheap estimator giving a coarse but monotone risk signal, which suffices because the conformal threshold τ—not the raw entropy—sets the accept/veto decision. The runtime rolls back when $u \ge \tau$ or a hard constraint is violated. Sweeping τ (full table in the supplement), the default $\tau = 0.55$ balances precision/recall at rollback precision/recall $\approx (0.86, 0.88)$ and a 2.8% anomaly rate (>85% of unsafe actions excluded before production); $\tau = 0.30$ raises recall to 0.94 (3.5% anomaly) and $\tau = 0.70$ raises precision to 0.91 (2.3% anomaly), so lower τ trades precision for recall.

Exploration Efficiency

We measure sample efficiency as the configurations needed to reach within 2% of the final median latency: SemantOS converges in 38–52 attempts versus 120–300 for BO/RL—a 2–4× speedup at equal *online* budget. This comparison favors SemantOS at deployment time: it front-loads cost into offline LLM fine-tuning and KB construction, whereas BO/RL starts cold. The speedup thus reflects amortizing domain knowledge across deployments, not a lower total (offline+online) cost (offline cost is reported in the artifact). Over a 30-day continuous deployment with repeated drift, anomaly rate stayed below 3% and rollback precision above 0.85; recalibration restored coverage after each drift and phased rollouts limited worst-case exposure, empirically supporting Theorem 1 and Proposition 1.

Human Factors

Expanded user study (N=32 SRE/DevOps). In a within-subjects study, 32 SRE/DevOps practitioners (5–15 years' experience) performed three tasks (incident classification, configuration selection, decision rationale) under four conditions: baseline, expert tuning, BO/RL, and SemantOS with explanations and safety. SemantOS reduced decision time vs. BO/RL by 28% ($p < 0.01$, Wilcoxon signed-rank) and cognitive load (NASA-TLX) by 18% vs. expert and 24% vs. BO/RL. All reported p-values survive Holm–Bonferroni correction across the 3×4 conditions; the study ran under an approved IRB protocol with informed consent and retained no PII. SemantOS also outperformed all comparison conditions on explanation usefulness and uncertainty-corrected reliability ratings (Cliff's $\delta \in [0.42, 0.58]$), with participants citing evidence tracking and provenance-validated explanations as decisive—indicating that the explainability and safety design reduces operator cognitive burden beyond raw performance gains.

Discussion

Modeling tunable interactions as a typed knowledge graph captures causal couplings that black-box optimizers miss; staged rollout with rollback prevents catastrophic regressions under workload shift; and adaptive weight decay suppresses stale correlations. Unlike black-box BO/RL and RL-only systems (e.g., AdaptiveKernel, OS-R1), SemantOS combines graph-grounded reasoning with conformal, uncertainty-gated safety and provenance-carrying explanations, yielding Pareto-superior trade-offs and faster *online* convergence under drift. Empirically the guarantees held: after each drift, recalibration restored coverage and rollback precision within the window predicted by Prop. 1. Extending these ideas to database, network-scheduling, and compiler optimization is promising.

Limitations. (i) *Exchangeability*: conformal coverage assumes within-window exchangeability; sustained fast drift can briefly violate this until recalibration completes (Prop. 1). (ii) *Residual LLM risk*: despite retrieval grounding, constrained decoding, and provenance checks, residual hallucination is non-zero, so the safety runtime—not a correctness proof—is the last line of defense. (iii) *Generality*: results cover Linux 6.4 on three server classes and six workload families; embedded, hard-real-time, and edge deployments where the 13B reasoner's cost (a 26 GB A100 side-car) dominates remain future work (Jeary et al. 2024; Correia et al. 2024).

Ethical and Responsible Deployment

SemantOS follows *least privilege*: automated writes occur only through designated control steps with rollback policies, immutable audit logs, and governance checkpoints. With a read-only default and safety-first staged rollout, automated kernel modifications stay safe and auditable. Experiments used only synthetic workloads and system-generated telemetry; the user study (N=32) ran under an approved IRB protocol with informed consent, collected no PII, and stored only de-identified task metrics.

Conclusion

SemantOS turns kernel tuning into a systematic control loop combining graph-grounded semantic reasoning, retrieval-augmented LLM inference, and conformal-calibrated safety. Across diverse workloads it reduces tail latency, anomaly rate, and operator effort while emitting provenance-grounded explanations; the same principles extend to database and network tuning.

References

Biswas, S.; She, Y.; and Kang, E. 2023. Towards Safe ML-Based Systems in Presence of Feedback Loops. In *Proceedings of the 1st International Workshop on Dependability and Trustworthiness of Safety-Critical Systems with Machine Learned Components (SE4SafeML '23)*, 18–21. San Francisco, CA, USA: ACM. .

Brown, T. B.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; Agarwal, S.; Herbert-Voss, A.; Krueger, G.; Henighan, T.; Child, R.; Ramesh, A.; Ziegler, D. M.; Wu, J.; Winter, C.; Hesse, C.; Chen, M.; Sigler, E.; Litwin, M.; Gray, S.; Chess, B.; Clark, J.; Berner, C.; McCandlish, S.; Radford, A.; Sutskever, I.; and Amodei, D. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 1877–1901. Curran Associates, Inc. .

Chen, H.; Wen, Y.; Cheng, L.; Kuang, S.; Liu, Y.; Li, W.; Li, L.; Zhang, R.; Song, X.; Li, W.; Guo, Q.; and Chen, Y. 2024. AutoOS: Make Your OS More Powerful by Exploiting Large Language Models. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, volume 235 of *Proceedings of Machine Learning Research*, 7511–7525. Vienna, Austria: PMLR. .

Correia, A. H. C.; Massoli, F. V.; Louizos, C.; and Behboodi, A. 2024. An Information Theoretic Perspective on Conformal Prediction. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. Curran Associates, Inc. .

Dean, J.; and Barroso, L. A. 2013. The Tail at Scale. *Communications of the ACM*, 56(2): 74–80. .

Fingler, H.; Tarte, I.; Yu, H.; Szekely, A.; Hu, B.; Akella, A.; and Rossbach, C. J. 2023. Towards a Machine Learning-Assisted Kernel with LAKE. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '23)*, 846–861. Vancouver, BC, Canada: ACM. .

Herodotou, H.; Chen, Y.; and Lu, J. 2021. A Survey on Automatic Parameter Tuning for Big Data Processing Systems. *ACM Computing Surveys*, 53(2): 1–37. .

Jeary, L.; Kuipers, T.; Hosseini, M.; and Paoletti, N. 2024. Verifiably Robust Conformal Prediction. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. Curran Associates, Inc. .

Jiang, J.; Wang, J.; Yan, Y.; Liu, Y.; Zhu, J.; Zhang, M.; and Gao, L. 2025. Do Large Language Models Excel in Complex Logical Reasoning with Formal Language? In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP 2025)*, 16878–16903. Suzhou, China: Association for Computational Linguistics. .

Lee, H.; Jung, S.; and Jo, H. 2022. STUN: Reinforcement-Learning-Based Optimization of Kernel Scheduler Parameters for Static Workload Performance. *Applied Sciences*, 12(14): 7072. .

Lee, S.; Sim, W.; Shin, D.; Seo, W.; Park, J.; Lee, S.; Hwang, S.; Kim, S.; and Kim, S. 2025. Reasoning Abilities of Large Language Models: In-Depth Analysis on the Abstraction and Reasoning Corpus. *ACM Transactions on Intelligent Systems and Technology*, 16(6): 1–52. .

Li, T.; Zhong, J.; Liu, J.; Wu, W.; and Zhang, C. 2018. Ease.ml: Towards Multi-Tenant Resource Sharing for Machine Learning Workloads. *Proceedings of the VLDB Endowment*, 11(5): 607–620. .

Lin, H.; Li, Y.; Luo, H.; Yao, K.; Zhang, L.; Xing, M.; and Wu, Y. 2025. OS-R1: Agentic Operating System Kernel Tuning with Reinforcement Learning. arXiv preprint arXiv:2508.12551. , arXiv:2508.12551.

Maas, M. 2020. A Taxonomy of ML for Systems Problems. *IEEE Micro*, 40(5): 8–16. .

Mao, H.; Schwarzkopf, M.; Venkatakrishnan, S. B.; Meng, Z.; and Alizadeh, M. 2019. Learning Scheduling Algorithms for Data Processing Clusters. In *Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM '19)*, 270–288. Beijing, China: ACM. .

Mialon, G.; Dessì, R.; Lomeli, M.; Nalmpantis, C.; Pasunuru, R.; Raileanu, R.; Rozière, B.; Schick, T.; Dwivedi-Yu, J.; Celikyilmaz, A.; Grave, E.; LeCun, Y.; and Scialom, T. 2023. Augmented Language Models: A Survey. arXiv preprint arXiv:2302.07842. , arXiv:2302.07842.

Mirhoseini, A.; Goldie, A.; Yazgan, M.; Jiang, J. W.; Songhori, E.; Wang, S.; Lee, Y.-J.; Johnson, E.; Pathak, O.; Nova, A.; Pak, J.; Tong, A.; Srinivasa, K.; Hang, W.; Tuncer, E.; Le, Q. V.; Laudon, J.; Ho, R.; Carpenter, R.; and Dean, J. 2021. A Graph Placement Methodology for Fast Chip Design. *Nature*, 594(7862): 207–212. .

Moser, R.; Rengarajan, S.; and Narayanamurthy, G. 2021. Decision Intelligence: Creating a Fit Between Intelligence Requirements and Intelligence Processing Capacities. *IIM Kozhikode Society & Management Review*, 10(2): 160–177. .

OpenAI. 2024. GPT-4 Technical Report. arXiv preprint arXiv:2303.08774. , arXiv:2303.08774.

Ousterhout, K.; Rasti, R.; Ratnasamy, S.; Shenker, S.; and Chun, B.-G. 2015. Making Sense of Performance in Data Analytics Frameworks. In *Proceedings of the 12th USENIX Conference on Networked Systems Design and Implementation (NSDI '15)*, 293–307. Oakland, CA, USA: USENIX Association. .

Roman, I.; Ceberio, J.; Mendiburu, A.; and Lozano, J. A. 2016. Bayesian Optimization for Parameter Tuning in Evolutionary Algorithms. In *Proceedings of the 2016 IEEE Congress on Evolutionary Computation (CEC)*, 4839–4845. Vancouver, BC, Canada: IEEE. .

Roman, I.; Santana, R.; Mendiburu, A.; and Lozano, J. A. 2019. An Experimental Study in Adaptive Kernel Selection for Bayesian Optimization. *IEEE Access*, 7: 184294–184302. .

Touvron, H.; Lavril, T.; Izacard, G.; Martinet, X.; Lachaux, M.-A.; Lacroix, T.; Rozière, B.; Goyal, N.; Hambro, E.; Azhar, F.; Rodriguez, A.; Joulin, A.; Grave, E.; and Lample, G. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv preprint arXiv:2302.13971. , arXiv:2302.13971.

Van Aken, D.; Pavlo, A.; Gordon, G. J.; and Zhang, B. 2017. Automatic Database Management System Tuning Through Large-Scale Machine Learning. In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD '17)*, 1009–1024. Chicago, IL, USA: ACM. .

Velez, M.; Jamshidi, P.; Sattler, F.; Siegmund, N.; Apel, S.; and Kästner, C. 2020. ConfigCrusher: Towards White-Box Performance Analysis for Configurable Systems. *Automated Software Engineering*, 27(3–4): 265–300. .